ScalaTropy: Multiparty Coordination with Monadic Communication Primitives

Nicolas Farabegoli[1], Luca Tassinari[1], Gianluca Aguzzi[1], and Mirko Viroli[1]

University of Bologna, Bologna, Italy
{nicolas.farabegoli,gianluca.aguzzi,mirko.viroli}@unibo.it
luca.tassinari10@studio.unibo.it

Abstract. Multiparty languages provide a concrete foundation for expressing complex coordination behaviors in a single, coherent specification. Based on this idea, several paradigms have been proposed in the literature over the years. Among them, choreographic programming is a widely adopted paradigm for defining deadlock-free distributed systems, while multitier programming takes a different approach, focusing on partitioning system logic across different execution tiers.
Recognizing that these paradigms share fundamental similarities as multiparty languages, we build on choreographic programming while importing static architectural descriptions and placement types inspired by ScalaLoci. We introduce ScalaTropy, a coordination language that integrates multiple communication schemes to establish both isotropic and anisotropic communication patterns, while incorporating placement types and type-level architecture specification inspired by ScalaLoci, advancing beyond the capabilities of traditional choreographic languages. We provide a Scala implementation leveraging monadic constructs, which cleanly separate the language specification from the underlying monadic effects that drive coordination mechanisms. Finally, we present an empirical evaluation demonstrating the language's expressiveness and an analysis of communication overhead.

Keywords: Multiparty languages · Tagless final · Choreographic Programming · Multitier programming

1 Introduction

Choreographic programming [21,1] and *multitier programming* [29] are two prominent paradigms in the landscape of multiparty languages [17]. They address the challenge of coordinating modern distributed systems—from microservice architectures to IoT networks—by providing high-level abstractions to define complex collective behaviors in a single, coherent codebase, abstracting away low-level details such as message passing and synchronization protocols.

Choreographic programming focuses on defining the interactions between multiple parties in a distributed system from a global perspective. It allows developers to describe the overall system as a "third-person narrative", specifying how each party should behave and interact with others. Typically, choreographic

languages provide high-level abstractions for defining which peer executes which part of the code, and for specifying communication patterns between peers, as well as specifying which peers are involved in the system. Over the years, a strong theoretical foundation has been built around choreographic programming [2], with formal models and verification techniques ensuring properties such as deadlock-freedom and communication safety [4,14]. These foundations have influenced a rich ecosystem of practical frameworks, from the Jolie service-oriented language [22] to choreographic tools ensuring correctness properties such as AIOCJ [12], and have been comprehensively systematized in [15].

A related approach is taken by multitier programming, which focuses on partitioning the system logic across different execution tiers. Differently from choreographic programming, multitier programming describes the distributed system as a relation between different tiers, without involving explicit communication primitives, but rather the communication is inferred from the placement of code across tiers. A common feature in multitier languages is the definition of an architecture, i.e., a set of peers and their relationships, which allows preventing unintended communications between peers not allowed by the architecture specification [27].

While both paradigms have proven effective in their respective domains, opportunities remain to enhance their communication models to address emerging practical requirements. On the one hand, choreographic languages typically focus on point-to-point communication patterns, with selective multicast communication to subsets of participants sometimes requiring additional constructs or workarounds, such as broadcast-style primitives combined with conditional filtering or defining restricted scopes like conclaves. For instance, in a distributed auction system, selectively sending updates only to bidders interested in a specific item may require broadcasting to all participants followed by filtering, which can introduce additional network traffic. On the other hand, multitier programming languages leverage implicit communication through shared placements: while this is an elegant approach for tier-crossing data flow, it can make explicit selective communication patterns less straightforward to express.

Recognizing that choreographic programming can benefit from richer architectural information, we extend choreographic programming with static architectural descriptions and placement types inspired by ScalaLoci, aimed at enriching overall expressiveness. To this end, this paper introduces ScalaTropy, a choreographic coordination language that integrates type-level architecture encoding via placement types (Section 3.1) with choreographic global specification (Section 3.3), and extends choreographic programming with differentiated communication primitives that support both isotropic (broadcast-style) and anisotropic (selective) communication patterns.

This paper provides the following key contributions:

- **Unified multiparty language design**: we present the coordination language ScalaTropy, which integrates choreographic global specification with ScalaLoci-inspired architectural constraints, enabling developers to express complex multiparty patterns while maintaining architectural discipline.

- **Differentiated communication schemes**: we introduce a communication model supporting both isotropic (broadcast-style) and anisotropic (selective) communication patterns, allowing developers to optimize information flow based on application requirements.
- **Type-level architecture enforcement**: inspired by ScalaLoci [28], we provide a programming interface to encode architectural constraints at the type level, preventing invalid communications between peers at compile time and ensuring adherence to the specified architecture.
- **Monadic implementation framework**: we provide a Scala implementation of ScalaTropy as a monad, using so-called "tagless final encoding" [6], which cleanly separates language semantics from execution effects, facilitating extensibility and integration with other effects.

We validate ScalaTropy through empirical evaluation by implementing representative use cases drawn from the choreographic and ScalaLoci literature, showing that in certain contexts our approach can reduce communication overhead and improve code maintainability compared to existing approaches. Additionally, we demonstrate that ScalaTropy's type system catches common distributed programming errors at compile time that would otherwise manifest as runtime failures. The full code of the framework along with the case studies presented in the paper are released as open source with a permissive license[1], and permanently archived on Zenodo [13].

The rest of the paper is structured as follows. Section 2 motivates the need for differentiated communication patterns in multiparty languages. Section 3 presents the design and features of ScalaTropy, while Section 4 and Section 5 describe its Scala implementation and empirical evaluation, respectively. Section 6 discusses related work in the field of multiparty languages. Finally, Section 7 concludes the paper and outlines future research directions.

2 Communication Patterns in Multiparty Languages

Before introducing the ScalaTropy language, we motivate the need for differentiated communication patterns in multiparty languages.

To establish a fair comparison, we evaluate our approach against frameworks that are similarly implemented as internal domain-specific languages (DSLs), matching the design of ScalaTropy. While more mature and feature-rich implementations of both paradigms exist [16], focusing on internal DSLs ensures a consistent baseline for analysis.

Since our goal is to enrich choreographic programming with explicit architectural descriptions, to make a fair comparison, we selected *HasChor* [26] as a representative choreographic language and *ScalaLoci* [28] as the source of the architectural typing discipline adopted in ScalaTropy. The former is a Haskell embedded DSL that leverages monadic constructs to express choreographies, sharing similarities with our (monadic) approach. The latter is a Scala embedded

[1] https://github.com/nicolasfara/scalatropy

DSL that introduces type-level architecture specification, which we also adopt in ScalaTropy. ScalaLoci is one of the most mature and popular multitier programming languages, providing strong static guarantees, and as it is implemented in Scala, it shares similarities with our Scala-based implementation of ScalaTropy.

Let us formally characterize the communication patterns supported by multiparty languages. Before defining the communication patterns, we introduce some notation inspired by the formalism used in [28].

With P we denote a peer type, which represents a class of devices in the distributed system. We define $\mathcal{A}$ as the architecture specification of the distributed system, composed of a set of peer types P and a set of tie relationships $\mathsf{P}_1 \mapsto \mathsf{P}_2$ between peers.

$$\mathcal{A} ::= \{\mathsf{P}\} \mid \mathcal{A} \cup \mathcal{A} \mid \mathsf{P}_1 \mapsto^1 \mathsf{P}_2 \mid \mathsf{P}_3 \mapsto^* \mathsf{P}_4$$

With $\mathsf{P}_1 \mapsto^1 \mathsf{P}_2$ we denote a tie relationship where peer type P_1 is tied to a single instance of peer type P_2, and with $\mathsf{P}_3 \mapsto^* \mathsf{P}_4$ we denote a tie relationship where peer type P_3 is tied to multiple instances of peer type P_4.

We further define as Ω the runtime peer configuration, composed of a set of peer instances $\pi^{\mathsf{P}}_{\mathsf{n}}$, where n is a unique identifier for the peer instance and P is the peer type of the instance, and a set of communication links $\pi^{\mathsf{P}_1}_{\mathsf{n}} \rightarrow \pi^{\mathsf{P}_2}_{\mathsf{m}}$ between peer instances. We omit the subscript of the peer instance when it is not relevant for the discussion.

Definition 1 (Architecture satisfaction). *A runtime peer configuration Ω satisfies an architecture specification $\mathcal{A}$, written as $\Omega \models \mathcal{A}$, when:*

- *For every peer type $\mathsf{P} \in \mathcal{A}$, there exists at least one peer instance $\pi^{\mathsf{P}}_{\mathsf{n}} \in \Omega$.*
- *For every tie relationship $\mathsf{P}_1 \mapsto^1 \mathsf{P}_2 \in \mathcal{A}$, there exists a single $\pi^{\mathsf{P}_1}_{\mathsf{n}} \rightarrow \pi^{\mathsf{P}_2}_{\mathsf{m}} \in \Omega$.*
- *For every tie relationship $\mathsf{P}_3 \mapsto^* \mathsf{P}_4 \in \mathcal{A}$, for every peer instance $\pi^{\mathsf{P}_3}_{\mathsf{n}} \in \Omega$, there exists at least one communication link $\pi^{\mathsf{P}_3}_{\mathsf{n}} \rightarrow \pi^{\mathsf{P}_4}_{\mathsf{m}} \in \Omega$.*

Definition 2 (Point-to-Point Communication). *Given $\Omega \models \mathcal{A}$ and peer instances $\pi^{\mathsf{P}_s}, \pi^{\mathsf{P}_r} \in \Omega$ with $\pi^{\mathsf{P}_s} \rightarrow \pi^{\mathsf{P}_r} \in \Omega$, a* point-to-point communication *transmits a message m from the sender instance π^{P_s} to the receiver instance π^{P_r}:*

$$\pi^{\mathsf{P}_s} \xrightarrow{m} \pi^{\mathsf{P}_r}$$

This is the most basic form of communication in multiparty languages (Figure 1a).

Definition 3 (Isotropic Communication). *Given $\Omega \models \mathcal{A}$ and peer instances $\pi^{\mathsf{P}_s}, \{\pi^{\mathsf{P}_r}_1, \ldots, \pi^{\mathsf{P}_r}_{\mathsf{n}}\} \subseteq \Omega$ with $\pi^{\mathsf{P}_s} \rightarrow \pi^{\mathsf{P}_r}_{\mathsf{i}} \in \Omega$, an* isotropic communication *transmits the same message m from the sender instance π^{P_s} to every receiver instance $\pi^{\mathsf{P}_r}_{\mathsf{i}}$:*

$$\pi^{\mathsf{P}_s} \xrightarrow{m} \{\pi^{\mathsf{P}_r}_1, \ldots, \pi^{\mathsf{P}_r}_{\mathsf{n}}\}$$

Isotropic communication (Figure 1b) models broadcast-style dissemination where every receiver obtains an identical copy of the payload, such as a leader broadcasting a configuration update to all replicas.

Definition 4 (Anisotropic Communication). *Given $\Omega \models \mathcal{A}$ and peer instances $\pi^{\mathsf{P_s}}, \{\pi_1^{\mathsf{P_r}}, \ldots, \pi_\mathsf{n}^{\mathsf{P_r}}\} \subseteq \Omega$ with $\pi^{\mathsf{P_s}} \rightarrow \pi_\mathsf{i}^{\mathsf{P_r}} \in \Omega$, an* anisotropic communication *transmits a potentially different message m_i from the sender instance $\pi^{\mathsf{P_s}}$ to each receiver instance $\pi_\mathsf{i}^{\mathsf{P_r}}$:*

$$\pi^{\mathsf{P_s}} \xrightarrow{m_i} \{\pi_1^{\mathsf{P_r}}, \ldots, \pi_\mathsf{n}^{\mathsf{P_r}}\}$$

Anisotropic communication (Figure 1c) generalises the isotropic case by allowing the sender to tailor each message to its recipient, for example when a coordinator assigns distinct tasks to individual workers.

Definition 5 (Co-anisotropic Communication). *Given $\Omega \models \mathcal{A}$ and peer instances $\{\pi_1^{\mathsf{P_s}}, \ldots, \pi_\mathsf{n}^{\mathsf{P_s}}\}, \pi^{\mathsf{P_r}} \subseteq \Omega$ with $\pi_\mathsf{i}^{\mathsf{P_s}} \rightarrow \pi^{\mathsf{P_r}} \in \Omega$, a* co-anisotropic communication *transmits a potentially different message m_i from each sender instance $\pi_\mathsf{i}^{\mathsf{P_s}}$ to the receiver instance $\pi^{\mathsf{P_r}}$:*

$$\{\pi_1^{\mathsf{P_s}}, \ldots, \pi_\mathsf{n}^{\mathsf{P_s}}\} \xrightarrow{m_i} \pi^{\mathsf{P_r}}$$

Co-anisotropic communication (Figure 1d) captures fan-in scenarios, such as multiple sensors reporting their readings to a single aggregator.

(a) Point-to-point (b) Isotropic (c) Anisotropic (d) Co-anisotropic

Fig. 1: Communication patterns in multiparty languages.

These communication patterns provide a comprehensive set of primitives for expressing various communication scenarios in multiparty languages.

2.1 Communication Operations in Other Languages

To provide an overview of the communication operations supported by the selected DSLs, Table 1 summarizes the primitives provided by HasChor, CloudChor, ScalaLoci, and ScalaTropy. We also include CloudChor [19] as an internal DSL which, unlike the basic point-to-point interactions in HasChor, provides support for differentiated communication patterns. From the table, we can see that HasChor only supports point-to-point communication, meaning that if we want to support, for instance, isotropic communication, we need to manually provide the logic to send the same message to multiple receivers, e.g., by invoking multiple point-to-point communications. ScalaLoci, on the other hand, supports both point-to-point and isotropic communication, allowing the receiver

Table 1: Communication operations supported by multiparty languages.

Comm. Types	HasChor	CloudChor	ScalaLoci	ScalaTropy
Point-to-point	~>	~>	asLocal*	comm
Isotropic	-	~>*	asLocalFromAllSeq*	isotropicComm
Anisotropic	-	~>.	V per P on R**	anisotropicComm
Co-anisotropic	-	*~>	V per R on P**	coAnisotropicComm

* We considered the functions that trigger the communication.
** Differentiated communications are expressed at the type level.

to extract the messages from all senders. It also supports anisotropic and co-anisotropic communication, however it requires boilerplate code to express these communication patterns, as can be seen in the following example.

```
val deployTask: Signal[Task] per Worker on Master =
  { worker: Remote[Worker] => Signal { assocs().get(worker) }}
```

To express anisotropic communication in ScalaLoci, developers must define a signal parameterized by the remote peer (`worker: Remote[Worker]`) and manually extract the appropriate message from an association map (`assocs().get(worker)`). While flexible, this approach requires boilerplate code and lacks the clarity of a dedicated communication primitive. In contrast, ScalaTropy provides native support for all communication patterns from Section 2 through dedicated primitives, simplifying multiparty coordination without sacrificing expressiveness.

3 The ScalaTropy DSL

Before introducing the characteristics of ScalaTropy, we present the main *choreographic* and *multitier* concepts that inspired our work.

3.1 Type-level Architecture Definition

Having a clear and type-safe definition of the architecture of a distributed system is a fundamental aspect of ScalaTropy. While several multiparty languages describe participants and communication structure in different ways, our work adopts the explicit type-level architecture description introduced by ScalaLoci [28], where developers declare peer families together with their admissible communication ties. In ScalaTropy, we import this ScalaLoci-inspired architectural layer into a choreographic setting, allowing developers to define a set of peers and their communication relationships. Inspired by ScalaLoci, we defined an architecture with three peers: `Client`, `Server`, and `Database`, where the `Client` can communicate with the `Server`, and the `Server` can communicate with the `Database`.

```
// Clients (peer) can communicate with a single Server (tie relationship)
type Client <: { type Tie <: Single[Server] }
// Servers can communicate with a single Database and multiple Clients
type Server <: { type Tie <: Single[Database] & Multiple[Client] }
// Database can communicate with one Server
type Database <: { type Tie <: Single[Server] }
```

The architecture above specifies which *class* of devices (i.e., peer types) are involved in the distributed system. To declare *instances* of these peers, we use the `type Client` syntax as defined in the architecture. To define *connections* (i.e., which peer can communicate with which other peer), we refine the peer definition specifying the `Client` to be a subtype (the `<:` in Scala refers to subtyping) of a structural type, i.e., an anonymous type with members defined inline, enclosed in curly braces. Such structural type defines a type member `Tie`, which specifies the communication relationships of the peer with other peers. In the case of `Client`, the `Tie` type member is defined as `Single[Server]`, indicating that the `Client` can communicate with a single instance of `Server`. When defining ties, we can use the *intersection type* operator `&` to specify multiple communication relationships, as shown in the definition of `Server`.

Ties Relationships Leveraging the high flexibility of Scala's type system, we can define the "structural signature" of a peer as `type Peer = { type Tie }`. Such a definition specifies a type alias to a structural type with a type member `Tie`. However, as such, the `Tie` type member is unconstrained, meaning that it can be defined arbitrarily. Therefore, to impose constraints on the `Tie` type member, we introduce two type constructors:

```
enum Quantifier[-P <: Peer]:
  case Single()
  case Multiple()
```

The `Quantifier` enum (i.e., an algebraic data type) introduces two type constructors: `Single` to denote a tie $\mathsf{P}_1 \mapsto^1 \mathsf{P}_2$ where a peer is tied to a single instance of another peer, and `Multiple` to denote a tie $\mathsf{P}_1 \mapsto^* \mathsf{P}_2$ where a peer is tied to multiple instances of another peer. However, with this definition we are only describing the *quantifier* of the tie relationship, but we are not actually specifying which peer types are involved in the tie.

To overcome this limitation, we introduced two types:

```
type TiedSingle[P <: Peer] = { type Tie <: Single[P] }
type TiedMultiple[P <: Peer] = { type Tie <: Multiple[P] }
```

The first one models a peer that is tied with a single instance of peer type `P`, while the second one models a peer that is tied with multiple instances of peer type `P`. Such types are useful to express constraints on communication patterns between peers, as we will see in the next sections.

3.2 Placement Types

Both *choreographic* and *multitier* programming use the concept of "values that live in the context of certain peer(s)"; in the former, it is often referred to as *located values* while in the latter, they are called *placed values*. Specifically, at the type level, we refine the value type with a notion of *placement*, indicating where the value is located in the distributed system.

In ScalaTropy, we reuse the syntax from ScalaLoci to define placement types. In Scala, we can define a value of type `Int` that is placed at the `Client` peer as `Int on Client`. The `on` type is an infix type which takes two type parameters: the first is the base type (here, `Int`), and the second is the peer where the value is placed (here, `Client`).

Intuitively, a value of type `V on P` should be read as "a value of type `V` owned by peer `P`". When a multiparty program executes `on[P]`, only peers of type `P` actually evaluate the computation and materialize the value locally; all other peers only keep a typed reference to that placed value. As a consequence, the value can be inspected with `take` only from within an `on[P]` context, while communication primitives are the mechanism that makes the value available at other placements.

Placement types are crucial in ScalaTropy, as they allow us to express where computations and data reside in the distributed system, effectively encoding the peer instances $\pi_{\mathsf{n}}^{\mathsf{P}}$ of the runtime peer configuration Ω (Definition 1) at the type level. In conjunction with the architecture definition, they enable the type system to enforce communication constraints between peers. This means that any attempt to communicate between peers not allowed by the architecture will result in a compile-time error, thus preventing potential runtime failures due to invalid communications.

3.3 Language Primitives

ScalaTropy provides two categories of operations: **placed computations** for defining peer-specific computations and extracting local values, and **communication primitives** for peer-to-peer communication with differentiated schemes, all enforcing architecture constraints at the type level. These operations are defined in a single language trait, `MultiParty[F[_]: Monad]`, which abstracts over the effect type `F[_]`. This trait requires a `Monad` constraint (via context bound syntax in Scala, namely `: Monad`) to support monadic composition and allow for various execution strategies, as detailed in Section 4. We now introduce the main primitives in detail.

Place and extract values. To define computations that execute on specific peers, we need a mechanism to specify the execution context (the peer) and a mechanism to access values placed at that context. In ScalaTropy, these capabilities are provided by the following primitives:

```
trait MultiParty[F[_]: Monad]:
  def on[L <: Peer, V](body: Label[L] ?=> F[V]): F[V on L]
  def take[L <: Peer](using Label[L])[V](placed: V on L): F[V]
  /* ... other primitives ... */
```

Specifically, the `on` function executes computations only at peer instances of type `L`. Its `body` parameter is a context function with an implicit `Label[L]` parameter, enabling peer-specific operations like `take`. The result is a placement type `V on L` wrapped in the effect `F[_]`.

The `take` function extracts values placed at peer type `L`, requiring an implicit `Label[L]` parameter available only within `on[L]` contexts. This ensures that only locally placed values can be extracted; attempting to extract remote values results in a compile-time error.

The following example demonstrates `on` and `take` together.

```
def example[F[_]: Monad](using MultiParty[F]): F[Int on Client] = for
  placedValue: String on Client <- on[Client] { "Hello, from Client!" }
  stringLength: Int on Client <- on[Client] { take(placedValue).map(_.length) }
yield stringLength
```

The `<-` operator acts as variable assignment in a monadic composition—namely, it is internally turned into a `flatMap` method call, namely, the bind monadic operation. First, we place a string at `Client` producing `String on Client`. Then, we extract it using `take`, compute its length, and place the result back at `Client`.

Point-to-point Communication. Following Definition 2, where a message m is transmitted from a single sender $\pi^{\mathsf{P_s}}$ to a single receiver $\pi^{\mathsf{P_r}}$, the `comm` function transmits values from source peer `S` to destination peer `R`.

```
trait MultiParty[F[_]: Monad]:
  /* ... other primitives ... */
  def comm[S <: TiedSingle[R], R <: TiedSingle[S], V](value: V on S): F[V on R]
```

The transmission is reflected in the type signature: the input is a value placed at the sender `V on S`, and the result is a value placed at the receiver `V on R`—this type-level encoding is also used for the other communication primitives. Crucially, this primitive enforces architectural compliance by constraining `S` and `R` to be mutually tied to a single instance via `TiedSingle`. This primitive definition is similar to the one found in Choral [16] and HasChor/CloudChor [26].

To illustrate point-to-point communication, consider a simple message exchange between two peers, `Alice` and `Bob`. We assume these peers are defined in the architecture with a mutual single-instance tie relationship, allowing them to exchange messages directly:

```
def pointToPointExample[F[_]: Monad](using MultiParty[F]): F[Unit] = for
  messageFromAlice <- on[Alice] { "Hello, Bob!" }
  messageAtBob <- comm[Alice, Bob](messageFromAlice)
  _ <- on[Bob]:
    take(messageAtBob).map(msg => println(s"Bob received: $msg"))
yield ()
```

Isotropic Communication. Recalling Definition 3, isotropic communication sends the same message m from one sender $\pi^{\mathsf{P_s}}$ to all receiver instances $\{\pi_1^{\mathsf{P_r}}, \ldots, \pi_n^{\mathsf{P_r}}\}$—a selective multicast to a specific peer class, distinct from indiscriminate broadcast.

```
trait MultiParty[F[_]: Monad]:
  /* ... other primitives ... */
  def isotropicComm[S <: TiedMultiple[R], R <: TiedSingle[S], V](v: V on S):
      F[V on R]
```

The function requires `S` tied to multiple `R` instances, and `R` tied to a single `S` instance.

The following example shows the **`Server`** communicating with multiple **`Client`**s. The **`isotropicComm`** function (line 3) shares `updateFromServer` from **`Server`** to all **`Client`** instances. Then, each value is extracted in each client's context through **`take`** (line 5).

```
1 def isotropicExample[F[_]: Monad](using MultiParty[F]): F[Unit] = for
2   updateFromServer <- on[Server] { "Update available." }
3   updateAtClients <- isotropicComm[Server, Client](updateFromServer)
4   _ <- on[Client]:
5     take(updateAtClients).map(update => println(s"Client received: $update"))
6 yield ()
```

Anisotropic Communication. Corresponding to the pattern defined in Definition 4, anisotropic communication allows sending potentially different messages m_i from a single sender $\pi^{\mathsf{P_s}}$ to each receiver $\pi_i^{\mathsf{P_r}}$, enabling per-receiver customization and reducing unnecessary communication overhead.

```
trait MultiParty[F[_]: Monad]:
  type Remote[P <: Peer]
  type Anisotropic[P <: Peer, V]
  /* ... other primitives ... */
  def anisotropicMessage[S <: TiedMultiple[R], R <: TiedSingle[S], V](
    using Label[S]
  )(value: Map[Remote[R], V], default: V): F[Anisotropic[R, V]]
  def anisotropicComm[S <: TiedMultiple[R], R <: TiedSingle[S], V](
    value: Anisotropic[R, V] on S
  ): F[V on R]
  def reachablePeers[R <: Peer]: F[NonEmptyList[Remote[R]]]
```

This pattern requires two functions: **`anisotropicMessage`** creates an **`Anisotropic`**`[R, V]` value mapping remote instances to messages (with a default), and **`anisotropicComm`** transmits it. The **`reachablePeers`** function returns all reachable instances of a given peer type. Together with the statically declared ties, this lets a choreography tailor messages to the concrete peer instances that are currently reachable and architecturally admissible.

The following example shows **`Server`** sending different updates to **`Client`**s.

```
def anisotropicExample[F[_]: Monad](using MultiParty[F]): F[Unit] = for
  updates: Anisotropic[Client, String] on Server <- on[Server]:
    anisotropicMessage[Server, Client, String](
      reachablePeers[Client].map(client -> s"Update for $client").toMap,
      "No Updates"
    )
  updateAtClients <- anisotropicComm[Server, Client](updates)
  _ <- on[Client]:
    take(updateAtClients).map(update => println(s"Client received: $update"))
yield ()
```

We create an anisotropic message mapping each **Client** to a custom update, then transmit it via **anisotropicComm**. Each **Client** extracts its specific update using **take**.

Co-anisotropic Communication. Dual to the anisotropic case (cf. Definition 5), co-anisotropic communication captures the fan-in scenario where multiple senders $\{\pi_1^{\mathsf{P}_s}, \ldots, \pi_n^{\mathsf{P}_s}\}$ transmit potentially different messages m_i to a single receiver π^{P_r}.

```
trait MultiParty[F[_]: Monad]:
  /* ... other primitives ... */
  def coAnisotropicComm[S <: TiedSingle[R], R <: TiedMultiple[S], V](
    value: V on S
  ): F[Anisotropic[S, V] on R]
  def takeAll[L <: Peer, R <: Peer](using Label[L])[V](
    placed: Anisotropic[R, V] on L
  ): F[Map[Remote[R], V]]
```

The **coAnisotropicComm** function collects messages from multiple `S` senders to a single `R` receiver, producing **Anisotropic**`[S, V]` **on** `R`. The **takeAll** function extracts individual messages as a map. The following example shows multiple **Client**s sending to a single **Server**.

```
def coAnisotropicExample[F[_]: Monad](using MultiParty[F]): F[Unit] = for
  messageFromClient <- on[Client] { "Hello, Server!" }
  messagesAtServer <- coAnisotropicComm[Client, Server](
    messageFromClient
  )
  allMessages <- on[Server] { takeAll(messagesAtServer) }
  _ <- on[Server]:
    allMessages.foreach { case (client, msg) =>
      println(s"Server received from $client: $msg")
    }
yield ()
```

Each **Client** sends via **coAnisotropicComm**, and **Server** aggregates all messages using **takeAll**.

4 Implementation

ScalaTropy is implemented as an embedded DSL in Scala, leveraging Scala's powerful type system and monadic abstractions to define the language semantics and execution model. Its type system allows us to encode the architectural constraints and communication patterns at the type level, ensuring that any architectural violation is caught at compile time, while keeping the syntax clean and lightweight.

4.1 Monadic Implementation

Distributed systems inherently involve various computational effects: network communication, asynchronous operations, error handling, and state management. To ensure modularity and composability, we adopt a monadic approach

combined with the *tagless final* [6] encoding style. Programs are parameterized by an abstract effect type `F[_]`, with semantics defined purely through operations on this abstraction, thereby decoupling language specification from concrete execution strategies.

This is particularly effective in multiparty scenarios where coordination primitives must compose seamlessly regardless of whether they execute locally, communicate over the network, or involve other side effects. In ScalaTropy, this translates to the `MultiParty[F[_]]` trait, whose primitives (e.g., `on`, `comm`, `isotropicComm`) are defined against the abstract effect `F[_]`. The bound `Monad` on `F[_]` ensures that we can compose these operations in a monadic style, facilitating sequencing and error handling. A production interpreter may instantiate `F` with `IO` backed by a real MQTT network, while a test interpreter may substitute a pure, in-memory effect, reusing the same program definition in both cases without modification. Compared to free monads, tagless final avoids the overhead of re-defining the language semantics for each interpreter, reducing error-prone boilerplate and improving maintainability. This design still ensures good testability—by allowing effect substitution for testing purposes—and maintainability, as the language semantics is defined in a single place, independent of the execution strategy.

In practice, the `MultiParty` effect is implemented by composing two lower-level effects: `Network`, which abstracts the underlying communication layer, and `Environment`, which encapsulates the peer's local execution context. The `MultiParty` interpreter accepts these effect handlers as dependencies, delegating network operations and context queries to them while orchestrating the high-level coordination logic.

4.2 Placement Types

The placement type implementation in ScalaTropy is rather straightforward, as it is expressed as an infix type constructor `on` that takes a base type and a peer type as parameters. Under the hood, the `on` type is implemented as an *algebraic data type* (ADT) with two type constructors: `Local` which contains a unique identifier for the value placed, and the value itself; and `Remote` which contains only the unique identifier of the value placed. Notably, the unique identifier is a deterministic value that is used to reference the placed value in the distributed system.

A placement type is created out of a placed computation through the `on[P]` function. At runtime, when the `on[P]` function is evaluated, the `P` peer is compared to the current peer type: if they match, the lambda body of the `on[P]` function is executed and the resulting value is wrapped in a `Local` constructor; otherwise, the `on[P]` skips the execution of the lambda body and returns a `Remote` indicating that the value is not available locally.

Any operation that deals with a placement type checks whether the value is local or remote, and acts accordingly. For instance, the `comm` function sends the value via network to the destination peer type `R` only if the value is local, otherwise it interacts with the network to retrieve the value from the remote

peer type `S`. The general principle is very close to the one provided in [3], where the *end-point-projection* (EPP) is performed at runtime.

4.3 Network Layer

The network layer is a dependency of the `MultiParty` interpreter, abstracting the underlying communication mechanism and providing a uniform interface for sending and receiving messages between peers. Its interface defines two main operations: (i) a `send` primitive that takes a message, its reference, and the destination peer type as parameters, and sends the message to the destination peer type; (ii) a `receive` primitive that takes a message reference and the source peer type as parameters, and waits for a message to be received from the source peer type with the given reference. To complete the network layer, we also need a primitive to retrieve the list of reachable peer instances of a given peer type, and the local peer instance identifier.

```
trait Network[F[_], LP <: Peer]:
  type Address[P <: Peer]

  val localAddress: Address[LP]
  def send[V, To <: Peer](value: V, ref: Reference, to: Address[To]): F[Unit]
  def receive[V, From <: Peer](ref: Reference, from: Address[From]): F[V]
  def alivePeersOf[RP <: Peer: PeerTag]: F[NonEmptyList[Address[RP]]]
```

The network layer is intended as a (monadic) capability, allowing a seamless integration with the `MultiParty` language—via the tagless final encoding—enabling multiple implementations of the network layer for different communication protocols like MQTT, HTTP, and gRPC.

Despite its simplicity, the network layer ensures high flexibility and extensibility, as we can implement `send` and `receive` primitives with different semantics (e.g., reliable vs unreliable, ordered vs unordered, etc.) and different underlying communication protocols, providing specific guarantees to the layer above (i.e., the `MultiParty` interpreter). The only constraint is imposed by the `alivePeersOf` primitive, which must return a non-empty list of peer instances, as the communication primitives in ScalaTropy assume that there is at least one reachable peer instance of the destination peer type. This deals well with dynamic network topologies, where peers can join and leave the network at runtime, as the `alivePeersOf` primitive is implemented to reflect the current state of the network. In this way, the static architecture fixes which peer families may communicate, while runtime peer discovery determines which concrete instances participate in each communication.

5 A taste of ScalaTropy

To assess the practical utility of ScalaTropy, we evaluate it across two dimensions: expressiveness, and communication overhead.

```
1  object MasterWorker:
2    type Master <: { type Tie <: Multiple[Worker] }
3    type Worker <: { type Tie <: Single[Master] }
4
5    case class Task(x: Int) { def compute: Int = x * x }
6
7    def masterWorkerProgram[F[_]: MonadThrow](using MultiParty[F]): F[Unit] = for
8      tasks <- on[Master]:
9        for
10         peers <- reachablePeers[Worker]
11         allocation = peers.map(_ -> Task(Random.nextInt(100))).toList.toMap
12         message <- anisotropicMessage[Master, Worker](allocation, Task(0))
13       yield message
14     taskOnWorker <- anisotropicComm[Master, Worker](tasks)
15     partialResult <- on[Worker]:
16       for
17         t <- take(taskOnWorker)
18         res <- t.compute.pure[F]
19       yield res
20     allResults <- coAnisotropicComm[Worker, Master](partialResult)
21     result <- on[Master] { takeAll(allResults).map(_.values.sum) }
22   yield ()
```

Fig. 2: Master-Worker implementation in ScalaTropy.

5.1 Case Studies

To showcase the expressiveness of ScalaTropy on patterns studied in both the ScalaLoci and choreographic programming literature, we consider two representative examples: a Master-Worker application [28] and a Replicated Key-Value Store [26].

Master-Worker application. We consider a Master-Worker application in which a Master peer dispatches tasks to multiple Worker peers, and then aggregates the results to produce a final result (see Figure 2).

From an architectural perspective, the `Master` peer is tied to multiple `Worker` peers, while each `Worker` peer is tied to a single `Master`. The application logic is implemented in the `masterWorkerProgram` function, parameterized by an effect type `F[_]` constrained by `MonadThrow` to handle potential errors gracefully, and a `MultiParty[F]` instance to access the language primitives.

Initially (lines 10-15), on the `Master` peer, the reachable `Worker` peers are discovered; a task is allocated to each, preparing an *anisotropic message* for transmission. Transmission is delegated to the `anisotropicComm` primitive, which sends the corresponding task from the `Master` to each `Worker`, ensuring each `Worker` receives only its allocated task. Next (lines 17-22), on the `Worker` side, the task is extracted, computed locally, and the partial result is sent back to the `Master` using `coAnisotropicComm`, which collects the partial results into an anisotropic message placed on the `Master`. By its semantics, `coAnisotropicComm` implements a barrier synchronization, blocking until all reachable `Worker` peers have computed and returned their results. Finally (lines 24-26), the `Master` extracts all partial results and reduces them to produce the final result.

This program can be instantiated on a concrete peer instance, provided a network implementation and a coherent effect type as presented hereafter.

```
object MasterNode extends IOApp.Simple:
  override def run: IO[Unit] =
    val mqttNetwork = MqttNetwork
      .localBroker[IO, Master](Config(appId = "masterworker"))
    ScalaTropy(masterWorkerProgram[IO]).projectedOn[Master](using mqttNetwork)
```

Replicated Key-Value Store. The second example we considered is a Key-Value client-server application, where multiple clients interact with a remote key-value store, maintained by a server, supporting `put` (store) and `get` (retrieve) operations. To ensure fault tolerance, the data store is replicated across multiple server instances, following a primary-backup replication scheme. In this setup, clients communicate with a primary server instance for their operations, while the primary server is responsible for forwarding `put` operations to the backup server nodes to maintain data consistency across replicas.

The architecture of this system therefore consists of three peer types: **`Client`**, **`Primary`**, and **`Backup`**. Each **`Client`** is tied to a single **`Primary`** server instance to which it sends its requests, while the **`Primary`** server is tied to multiple **`Backup`** server instances and multiple **`Client`** peers. **`Backup`** nodes are tied to a single **`Primary`** server instance from which they receive updates. Responses and requests are encoded as ADTs as shown in the following code snippet.

```
object KeyValueStoreChoreo:
  type Client <: { type Tie <: Single[Primary] }
  type Primary <: { type Tie <: Multiple[Backup] & Multiple[Client] }
  type Backup <: { type Tie <: Single[Primary] }

  enum Request:
    case Get(key: String)
    case Put(key: String, value: String)
    case Empty

  enum Response:
    case Value(value: Option[String])
    case Ack
    case Empty
```

The code snippet in Figure 3 shows a synchronous implementation of the Key-Value client-server application: the **`Primary`** server applies writing operations, gathers acknowledgments from the **`Backup`** nodes, and then sends responses back to the clients.

First, at line 5, **`Client`** peers wait for a request (coming from user's input or other sources). Next, at line 6, each **`Client`** sends its request to the **`Primary`** server instance it is connected to, leveraging the **`coAnisotropicComm`** primitive. Server-side, once all requests are received, they are unpacked, processed by the **`Primary`** server's local storage, and the corresponding responses are prepared for anisotropic communication back to the **`Client`** peers (lines 8-14). Before sending, on the **`Primary`** server, all `Put` requests are extracted and forwarded to the **`Backup`** server instances using isotropic communication (lines 16-20). This ensures each **`Backup`** node receives and consistently applies the client operations to its replica

```
def kvs[F[_]: {Sync, Console}](
  primaryStorage: KeyValueStore[F, String, String] on Primary,
  backupStorage: KeyValueStore[F, String, String] on Backup,
)(using MultiParty[F]): F[Unit] = for
  requestOnClient <- on[Client](waitForRequest)
  requestsOnPrimary <- coAnisotropicComm[Client, Primary](requestOnClient)
  responsesOnPrimary <- on[Primary]:
    for
      store <- take(primaryStorage)
      requests <- takeAll(requestsOnPrimary)
      responses <- store processAll requests
      message <- anisotropicMessage[Primary, Client](responses, Empty)
    yield message
  putRequests <- on[Primary]:
    takeAll(requestsOnPrimary).map(_.values.collect { case r: Put => r })
  requestsOnBackups <- isotropicComm[Primary, Backup](putRequests)
  ack <- on[Backup]:
    for
      store <- take(backupStorage)
      requests <- take(requestsOnBackups)
      _ = requests.foreach(request => store process request)
    yield Ack
  _ <- coAnisotropicComm[Backup, Primary](ack)
  responseOnClient <- anisotropicComm[Primary, Client](responsesOnPrimary)
  _ <- on[Client]:
    take(responseOnClient) flatMap (response => F.println(s"> $response"))
  _ <- kvs(primaryStorage, backupStorage)
yield ()
```

Fig. 3: Key-Value Store implementation in ScalaTropy.

storage. Finally, when all the **Backup** nodes have acknowledged the updates, the **Primary** server sends each response back to the corresponding **Client** using the **anisotropicComm** primitive (lines 22-24). Computation continues recursively, allowing the system to process multiple requests over time.

By adopting an architectural definition, together with architecture-aware communication primitives, the above implementation is guaranteed to work correctly with any number of backup replica servers. In contrast, traditional choreographic implementations require the number of backup instances to be known in advance, and the communication logic to be explicitly defined for each instance.

5.2 Selective Communication Advantages

ScalaTropy's support for selective communication primitives serves three main purposes: (i) reducing unnecessary communication overhead, (ii) enhancing security by preventing information leakage to unintended recipients, and (iii) providing richer expressiveness.

Communication efficiency. Selective communication primitives enable the sender to transmit to each peer only the information that they require to accomplish their tasks. This avoids the overhead associated with broadcasting messages to all participants and then filtering them at the receiver side, proving to be par-

ticularly beneficial in scenarios where the number of peers is large or when the total message size is significant.

To quantify the benefits of this approach, we consider a variant of the Master-Worker application discussed in Section 5, for computing the matrix-vector product over large inputs. This represents a canonical data-parallel workload, where the Master partitions the matrix row-wise and distributes distinct row blocks to each Worker, enabling independent computation of the corresponding partial results. By introducing a dedicated primitive for anisotropic communication, ScalaTropy avoids suboptimal communication patterns, such as broadcasting the entire matrix to all Workers, which would then filter out their relevant rows. This prevents significant communication overhead as the matrix size and number of Workers grow. Figure 4 presents this communication overhead for a 50×50 representative matrix, plotting the total data transmitted by the Master (or, equivalently, the cumulated data received by the Workers) for the broadcast and the anisotropic communication approaches, as the number of Workers increases. Both were implemented in ScalaTropy, where the broadcast variant simply replaces `anisotropicComm` with `isotropicComm`, making them directly comparable within the same framework. Data transmitted is measured in kilobytes (KB) at the network level; the results are independent of the underlying network protocol or deployment setup. As expected, while the broadcast approach incurs a linear increase in communication overhead with the number of Workers, the selective communication approach maintains almost constant overhead, with only a marginal increase due to per-Worker metadata and the distribution of the vector to be multiplied.

Security. Beyond efficiency, selective communication primitives contribute to data confidentiality at the communication level: by construction, each peer receives only the information necessary to perform its task, reducing the risk of unintended information disclosure compared to broadcast-based approaches.

Expressiveness. Lastly, first-class support for selective communication patterns allows developers to express their intent more clearly in the code, avoiding the opaque communication mechanisms required by existing multitier languages, such as the ScalaLoci pattern discussed in Section 2.

6 Related Work

This work sits at the intersection of choreographic programming and multitier programming, two paradigms that share the common goal of simplifying distributed system development through high-level abstractions. Although session types represent a relevant line of research for multiparty communication, with several notable implementations such as Scribble [18], Ferrite [7], we do not discuss them in detail here, as our approach focuses on the integration of choreographic and multitier paradigms rather than on session-typed communication channels.

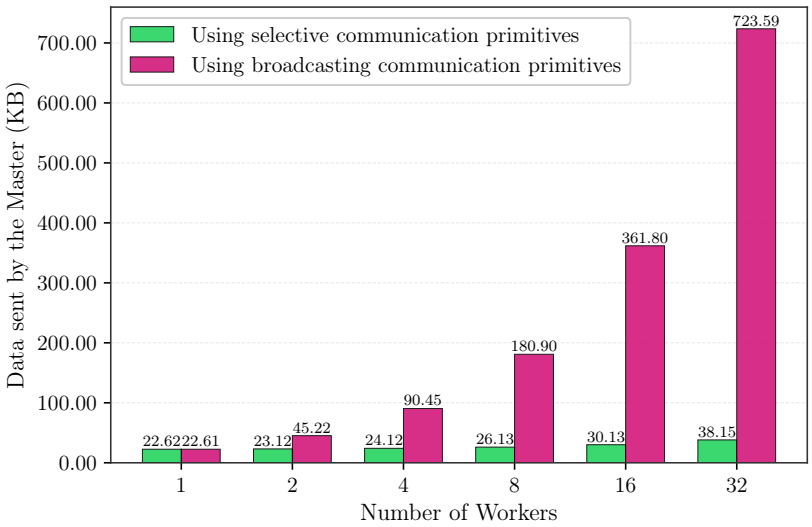

Fig. 4: Communication overhead comparison between broadcast and selective communication in a matrix-vector product application.

Choreographic Programming. Choreographic programming formalizes distributed coordination through global specifications [4]. On the practical side, the Jolie language [22] pioneered service-oriented coordination and influenced choreographic frameworks such as AIOCJ [12], which supports safe runtime adaptation. More recent implementations target specific host languages: Choral [16] is a Java-based external DSL; HasChor [26] embeds choreographies in Haskell using monadic constructs; and ChoRus [20] brings choreographic programming to Rust. These approaches primarily use point-to-point or broadcast communication, which can introduce overhead for selective multicast patterns. Procedural Choreographies [11] formalizes this setting through a connection graph, while also supporting unbounded process creation and name mobility. In [10], real-world computational algorithms are showcased using this language model. CloudChor [19] extends HasChor with multicast primitives for selective dissemination but lacks architectural constraints. Like HasChor and CloudChor, ScalaTropy uses monadic semantics for effect integration, but additionally enforces architectural constraints at the type level.

Multitier Programming. The Multi-Tier Calculus [23] formalizes tier-crossing computations but provides no concrete implementation. Practical languages include Ur/Web [8], Links [9], and Hop [25], which unify client and server code but rely on implicit communication through shared placements. ScalaLoci [28] provides type-level architecture specification and enforcement, ensuring communications respect the declared architecture at compile time. However, selective communication patterns in ScalaLoci require composition of lower-level primi-

tives, which can increase syntactic overhead compared to dedicated first-class constructs. ScalaTropy imports ScalaLoci's type-level architectural descriptions into a choreographic language with explicit selective communication primitives.

7 Conclusion

We presented ScalaTropy, a choreographic language for multiparty coordination that integrates ScalaLoci-inspired architectural descriptions in a single, coherent framework. We introduced explicit communication primitives for point-to-point, isotropic, anisotropic, and co-anisotropic communication patterns, enabling developers to express their intent clearly and efficiently while ensuring architectural compliance through type-level constraints. The monadic implementation of ScalaTropy allows for flexible execution strategies and seamless integration of effects, while the placement type system ensures that values are accessed only in their intended contexts.

We evaluated ScalaTropy through representative case studies, demonstrating its expressiveness and efficiency in implementing common distributed coordination patterns. Moreover, we analyzed the communication overhead of selective communication primitives, showing their advantages over traditional broadcast approaches in terms of efficiency and security.

Future work includes formalizing the semantics of ScalaTropy and proving properties such as type safety and deadlock freedom, as typical in the choreographic programming literature, and architectural compliance and communication correctness for the ScalaLoci-inspired architectural layer adopted by the language. Another direction is to investigate how existing choreographic languages such as Choral and HasChor could benefit from similar architectural descriptions and runtime role-instance information.

References

1. Ancona, D., Bono, V., Bravetti, M., Campos, J., Castagna, G., Deniélou, P., Gay, S.J., Gesbert, N., Giachino, E., Hu, R., Johnsen, E.B., Martins, F., Mascardi, V., Montesi, F., Neykova, R., Ng, N., Padovani, L., Vasconcelos, V.T., Yoshida, N.: Behavioral types in programming languages. Found. Trends Program. Lang. **3**(2-3), 95–230 (2016). https://doi.org/10.1561/2500000031, https://doi.org/10.1561/2500000031
2. Barbanera, F., Lanese, I., Tuosto, E.: A theory of formal choreographic languages. Log. Methods Comput. Sci. **19**(3) (2023). https://doi.org/10.46298/LMCS-19(3:9)2023, https://doi.org/10.46298/lmcs-19(3:9)2023
3. Bates, M., Kashiwa, S., Jafri, S., Shen, G., Kuper, L., Near, J.P.: Efficient, portable, census-polymorphic choreographic programming. Proc. ACM Program. Lang. **9**(PLDI) (Jun 2025). https://doi.org/10.1145/3729296, https://doi.org/10.1145/3729296
4. Carbone, M., Montesi, F.: Deadlock-freedom-by-design: multiparty asynchronous global programming. SIGPLAN Not. **48**(1), 263–274 (Jan 2013). https://doi.org/10.1145/2480359.2429101, https://doi.org/10.1145/2480359.2429101

5. Carbone, M., Montesi, F.: Deadlock-freedom-by-design: multiparty asynchronous global programming. In: Giacobazzi, R., Cousot, R. (eds.) The 40th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '13, Rome, Italy - January 23 - 25, 2013. pp. 263–274. ACM (2013). https://doi.org/10.1145/2429069.2429101, https://doi.org/10.1145/2429069.2429101
6. Carette, J., Kiselyov, O., Shan, C.: Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages. J. Funct. Program. **19**(5), 509–543 (2009). https://doi.org/10.1017/S0956796809007205, https://doi.org/10.1017/S0956796809007205
7. Chen, R.F., Balzer, S., Toninho, B.: Ferrite: A Judgmental Embedding of Session Types in Rust. In: Ali, K., Vitek, J. (eds.) 36th European Conference on Object-Oriented Programming (ECOOP 2022). Leibniz International Proceedings in Informatics (LIPIcs), vol. 222, pp. 22:1–22:28. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2022). https://doi.org/10.4230/LIPIcs.ECOOP.2022.22, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2022.22
8. Chlipala, A.: Ur/web: A simple model for programming the web. SIGPLAN Not. **50**(1), 153–165 (Jan 2015). https://doi.org/10.1145/2775051.2677004, https://doi.org/10.1145/2775051.2677004
9. Cooper, E., Lindley, S., Wadler, P., Yallop, J.: Links: Web programming without tiers. In: de Boer, F.S., Bonsangue, M.M., Graf, S., de Roever, W.P. (eds.) Formal Methods for Components and Objects, 5th International Symposium, FMCO 2006, Amsterdam, The Netherlands, November 7-10, 2006, Revised Lectures. Lecture Notes in Computer Science, vol. 4709, pp. 266–296. Springer (2006). https://doi.org/10.1007/978-3-540-74792-5_12, https://doi.org/10.1007/978-3-540-74792-5_12
10. Cruz-Filipe, L., Montesi, F.: Choreographies in Practice, p. 114–123. Springer International Publishing (2016). https://doi.org/10.1007/978-3-319-39570-8_8, http://dx.doi.org/10.1007/978-3-319-39570-8_8
11. Cruz-Filipe, L., Montesi, F.: Procedural Choreographic Programming, p. 92–107. Springer International Publishing (2017). https://doi.org/10.1007/978-3-319-60225-7_7, http://dx.doi.org/10.1007/978-3-319-60225-7_7
12. Dalla Preda, M., Gabbrielli, M., Giallorenzo, S., Lanese, I., Mauro, J.: Dynamic choreographies: Theory and implementation. Logical Methods in Computer Science **13** (2017)
13. Farabegoli, N., Tassinari, L., Aguzzi, G.: nicolasfara/scalatropy: Release v1.1.4 (Apr 2026). https://doi.org/10.5281/zenodo.19407062, https://doi.org/10.5281/zenodo.19407062
14. Ghilezan, S., Jaksic, S., Pantovic, J., Scalas, A., Yoshida, N.: Precise subtyping for synchronous multiparty sessions. J. Log. Algebraic Methods Program. **104**, 127–173 (2019). https://doi.org/10.1016/J.JLAMP.2018.12.002, https://doi.org/10.1016/j.jlamp.2018.12.002
15. Giallorenzo, S., Montesi, F., Gabbrielli, M.: Applied choreographies. In: International Conference on Formal Techniques for Distributed Objects, Components, and Systems. pp. 21–40. Springer (2018)
16. Giallorenzo, S., Montesi, F., Peressotti, M.: Choral: Object-oriented choreographic programming. ACM Trans. Program. Lang. Syst. **46**(1) (Jan 2024). https://doi.org/10.1145/3632398, https://doi.org/10.1145/3632398
17. Giallorenzo, S., Montesi, F., Peressotti, M., Richter, D., Salvaneschi, G., Weisenburger, P.: Multiparty languages: The choreographic and multitier cases (pearl).

In: Møller, A., Sridharan, M. (eds.) 35th European Conference on Object-Oriented Programming, ECOOP 2021, Aarhus, Denmark (Virtual Conference), July 11-17, 2021. LIPIcs, vol. 194, pp. 22:1–22:27. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021). https://doi.org/10.4230/LIPICS.ECOOP.2021.22, https://doi.org/10.4230/LIPIcs.ECOOP.2021.22

18. Honda, K., Mukhamedov, A., Brown, G., Chen, T., Yoshida, N.: Scribbling interactions with a formal foundation. In: Natarajan, R., Ojo, A.K. (eds.) Distributed Computing and Internet Technology - 7th International Conference, ICDCIT 2011, Bhubaneshwar, India, February 9-12, 2011. Proceedings. Lecture Notes in Computer Science, vol. 6536, pp. 55–75. Springer (2011). https://doi.org/10.1007/978-3-642-19056-8_4, https://doi.org/10.1007/978-3-642-19056-8_4
19. Ionescu, A., Russo, A.: Towards lightweight and efficient choreographic cloud services. In: Kameyama, Y., Xie, N. (eds.) Proceedings of the 2026 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation, PEPM 2026, Rennes, France, January 11-17, 2026. pp. 45–60. ACM (2026). https://doi.org/10.1145/3779209.3779537, https://doi.org/10.1145/3779209.3779537
20. Kashiwa, S., Kuper, L.: Chorus: Library-level choreographic programming in rust. Choreographic Programming (CP). Available at https://users. soe. ucsc. edu/~ lkuper/papers/chorus-cp24. pdf (2024)
21. Montesi, F.: Choreographic programming (2014)
22. Montesi, F., Guidi, C., Zavattaro, G.: Service-oriented programming with jolie. In: Bouguettaya, A., Sheng, Q.Z., Daniel, F. (eds.) Web Services Foundations, pp. 81–107. Springer (2014). https://doi.org/10.1007/978-1-4614-7518-7_4, https://doi.org/10.1007/978-1-4614-7518-7_4
23. Neubauer, M., Thiemann, P.: From sequential programs to multi-tier applications by program transformation. In: Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. p. 221–232. POPL '05, Association for Computing Machinery, New York, NY, USA (2005). https://doi.org/10.1145/1040305.1040324, https://doi.org/10.1145/1040305.1040324
24. Neubauer, M., Thiemann, P.: From sequential programs to multi-tier applications by program transformation. SIGPLAN Not. **40**(1), 221–232 (Jan 2005). https://doi.org/10.1145/1047659.1040324, https://doi.org/10.1145/1047659.1040324
25. Serrano, M., Gallesio, E., Loitsch, F.: Hop: a language for programming the web 2.0. In: Tarr, P.L., Cook, W.R. (eds.) Companion to the 21th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2006, October 22-26, 2006, Portland, Oregon, USA. pp. 975–985. ACM (2006). https://doi.org/10.1145/1176617.1176756, https://doi.org/10.1145/1176617.1176756
26. Shen, G., Kashiwa, S., Kuper, L.: Haschor: Functional choreographic programming for all (functional pearl). Proc. ACM Program. Lang. **7**(ICFP) (Aug 2023). https://doi.org/10.1145/3607849, https://doi.org/10.1145/3607849
27. Weisenburger, P., Salvaneschi, G.: Implementing a language for distributed systems: Choices and experiences with type level and macro programming in scala. Art Sci. Eng. Program. **4**, 17 (2020). https://doi.org/10.22152/programming-journal.org/2020/4/17
28. Weisenburger, P., Köhler, M., Salvaneschi, G.: Distributed system development with scalaloci. Proc. ACM Program. Lang. **2**(OOPSLA), 129:1–129:30 (2018). https://doi.org/10.1145/3276499, https://doi.org/10.1145/3276499
29. Weisenburger, P., Wirth, J., Salvaneschi, G.: A survey of multitier programming. ACM Comput. Surv. **53**(4), 81:1–81:35 (2021). https://doi.org/10.1145/3397495, https://doi.org/10.1145/3397495